

Relatorio de Seguranca - OAB de Bolso

Data: 2026-06-29

Testes realizados via navegador (F12 / DevTools), com conta de usuario comum.

Resumo executivo

Foram identificadas falhas criticas relacionadas ao endpoint POST /api/query, que expoe uma interface generica de banco de dados (ORM) ao cliente, sem limite de resultados e sem restricao de operacoes de escrita/exclusao. Adicionalmente, foi identificado que o site web.oabdebolso.com expoe codigo-fonte legivel (nao ofuscado) contendo schema completo do banco de dados (58 modelos).

Achados criticos

1. Endpoint /api/query permite extracao completa de dados sem paginacao

- **Endpoint:** POST https://web.oabdebolso.com/api/query
- **Descricao:** o endpoint aceita modelName, functionName: "findAll" e options livremente definidos pelo cliente, sem limite maximo de resultados nem whitelist de campos.
- **Evidencia:** requisicao com {"modelName":"Question", "functionName":"findAll", "options":{"attributes":["id"]}} retornou **61.490 registros** em uma unica chamada.
- **Impacto:** qualquer usuario autenticado (conta comum, paga) pode extrair integralmente o banco de questoes e toda a estrutura de curadoria de conteudo (disciplinas, objetivos, assuntos).
- **Modelos de conteudo confirmados extraiveis:**
 - Discipline: 57 registros
 - Objective: 142 registros
 - Subject: 3.775 registros
 - Question: 61.490 registros (enunciado completo incluso)

2. Endpoint /api/query aceita operacoes de escrita/exclusao (update, destroy)

- **Descricao:** o mesmo endpoint nao restringe functionName apenas a leitura. Testes com functionName: "destroy" e functionName: "update" usando um ID inexistente foram **aceitos pelo servidor** (nao rejeitados como operacao proibida).
- **Evidencia:**
 - destroy em ID falso retornou {"data":"","ok":true} (processado normalmente).
 - update em ID falso retornou erro 500 por formatacao do filtro ("Missing where attribute"), e nao por restricao de acesso a operacao - ou seja, o servidor tentou processar o update.
- **Impacto:** qualquer usuario autenticado pode, em tese, **alterar ou apagar registros reais em producao** (questoes, dados de progresso, e potencialmente qualquer modelo acessivel pelo endpoint).

- **Observacao:** nenhum dado real foi alterado ou apagado durante os testes. Foram usados apenas IDs inexistentes para validar o comportamento do endpoint sem causar dano.

3. Codig0-fonte e schema de banco de dados expostos em `web.oabdebolso.com`

- **Descricao:** o site web.oabdebolso.com serve uma aplicacao React cujo bundle JavaScript (main.[hash].chunk.js) contem definicoes de schema do banco de dados (Sequelize) e logica de negocio legivel (nao ofuscada), aparentemente por erro de build que incluiu codigo de backend no pacote do cliente.

- **Evidencia:** trecho exposto publicamente:

```
t.name="ClosedQuestion", t.plural="ClosedQuestions",  
t.schema=function(e){return{id:{type:e.UUID,primaryKey:!0},  
sequential:{type:e.INTEGER}}},  
t.associate=function(e){this.belongsTo(e.Question)}
```

- **58 modelos de banco de dados identificados no bundle publico,** incluindo tabelas sensiveis:

User, UserGoogleCredential, UserFacebookCredential, UserEmailCredential, PasswordResetToken, Permission, Role, RoleCategory, AccessRule, Installation, Device, Login, alem dos modelos de conteudo (Question, Subject, Discipline, Objective, etc).

- **Impacto combinado com o Achado 2:** ao conhecer o nome de todos os 58 modelos, o risco do endpoint /api/query se estende potencialmente a tabelas de usuarios, permissoes e credenciais - nao apenas ao conteudo de questoes.

- **Nao testado deliberadamente:** extracao/alteracao de dados pessoais de usuarios (modelos User e relacionados) nao foi realizada, para evitar exposicao de dados pessoais de terceiros (LGPD). Recomenda-se que a propria equipe tecnica valide esse ponto internamente.

Achados de severidade media/baixa

4. CORS configurado como totalmente aberto

- **Header observado:** Access-Control-Allow-Origin: * em web.oabdebolso.com/api.

- **Impacto:** mitigado pela exigencia de token valido, mas nao segue boas praticas. Recomenda-se restringir a origens conhecidas (app.oabdebolso.com, www.oabdebolso.com, etc).

5. Erro de configuracao no Firebase Storage

- **Erro retornado:** "A required service account is missing necessary permissions" ao acessar o bucket de Storage.

- **Impacto:** nao expoe dados, mas pode causar falhas em upload/download de arquivos no app. Requer reconexao da service account no Firebase Console.

Pontos testados e confirmados como seguros

- Cloud Firestore: desabilitado no projeto, sem exposicao.
- Realtime Database: desativado, sem exposicao.
- Autenticacao da API: todas as rotas testadas exigem token JWT valido (401 sem token).
- Validacao de token: tokens malformados/invalidos sao rejeitados corretamente.
- IDOR (acesso a dados de outro usuario trocando ID): bloqueado - a API valida que o `filter[user]` corresponde ao usuario do token.
- Codigo-fonte do aplicativo principal (`app.oabdebolso.com`, Flutter): devidamente ofuscado pelo compilador `dart2js`. Nao ha source map exposto (testado e confirmado por comparacao byte-a-byte com resposta de URL inexistente). Logica de negocio interna nao e legivel.
- Armazenamento local de credenciais (`flutter_secure_storage`): criptografado no navegador, nao exposto em texto puro.

Recomendacoes prioritarias

1. **Urgente:** restringir ou desativar o endpoint `/api/query` em producao. Substituir por endpoints especificos, com paginacao obrigatoria, whitelist de modelos/campos permitidos, e sem aceitar `functionName` de escrita (`update`, `destroy`, `create`) vindos do cliente sem controle de autorizacao adequado por modelo.
2. **Urgente:** investigar e corrigir o build de `web.oabdebolso.com` para garantir que nenhum codigo/schema de backend seja empacotado no bundle do cliente.
3. Auditar logs de acesso (se existirem) ao `/api/query` em busca de uso anormal (grandes volumes de `findAll`, ou qualquer uso de `update/destroy`) para identificar se a falha ja foi explorada por terceiros.
4. Restringir CORS a dominios conhecidos.
5. Corrigir a configuracao da service account do Firebase Storage.
6. Considerar implementar rate limiting mais agressivo e monitoramento de uso anormal por conta de usuario.

Metodologia

Todos os testes foram realizados via navegador (Google Chrome DevTools), utilizando uma conta de usuario comum, sem qualquer tentativa de acesso a sistemas ou contas de terceiros. Nenhum dado real foi alterado ou excluido durante os testes. Identificadores de autenticacao (tokens) foram omitidos deste relatorio por seguranca.